



# Compilers

**Guillermo A. Pérez**

**Lecture 5: Bottom-up parsing, CFSM and LR(0) parsers**

# TL;DR: This lecture in short

## How do we go from grammars to a deterministic parser?

We will try to do so by applying the grammar rules **in reverse**!

## Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

# Required and target competences

## What tools do we need?

Formal language theory (languages and machines, machines and computability);

## What skills will we obtain?

- theory: canonical finite automaton
- practice: how to implement LR parsers

## How will these skills be useful?

- It will allow you to understand the implementation of (older/other) parser generators.

# Outline

1. Bottom-up parsers
2. The non-deterministic PDA implementation
3. Viable prefixes
4. The canonical finite-state machine
5. LR(0) parsers

# Differences with top-down (1/2)

## Top-down parsing

Remember: Such parsers have two main actions they can perform.

- **Produce:** use a rule  $A \rightarrow \alpha$  to pop  $A$  from the stack and push  $\alpha$ .
- **Match:** pop a terminal  $a$  from the stack while simultaneously reading  $a$  from the input.

They start with  $S$  in the stack and accept by empty stack. In this way, a top-down parser unravels a derivation tree from top to bottom and left to right (i.e. following a left-most derivation).

# Differences with top-down (2/2)

## Bottom-up parsing

These work in the **opposite** way. They can perform two actions.

- **Shift**: read a terminal  $a$  from the input and push it into the stack.
- **Reduce**: use a rule  $A \rightarrow \alpha$  to pop the **handle**  $\alpha^R$  from the stack and push  $A$  instead.

They start with an empty stack and accept if the stack contents are  $S$ . In this way, a bottom-up parser unravels a derivation tree bottom to top and right to left! It builds a **right-most derivation in reverse order**.

## Example: bottom-up parsing

|     |     |               |           |
|-----|-----|---------------|-----------|
| (1) | Exp | $\rightarrow$ | Exp + Exp |
| (2) |     | $\rightarrow$ | Exp * Exp |
| (3) |     | $\rightarrow$ | (Exp)     |
| (4) |     | $\rightarrow$ | Id        |
| (5) |     | $\rightarrow$ | Cst       |

- Give a right-most derivation for  $\text{Id} + \text{Id} * \text{Id}$
- Simulate a correct bottom-up parsing of the string

# From a CFG to a bottom-up parser

## The PDA

- **Shift:** For all terminals  $a$ , the PDA has a transition that reads  $a$  and pushes  $a$  into the stack.
- **Reduce:** For every rule  $A \rightarrow \alpha_1 \dots \alpha_n$  the PDA has
  - states  $(A, \alpha_1, \dots, \alpha_i)$  for all  $1 \leq i < n$  and  $(A, \varepsilon)$ ,
  - transitions that pop letters from  $\alpha^R$  from the stack without reading from the input, and
  - a transition from  $(A, \varepsilon)$  to push  $A$  without reading from the input.
- The PDA accepts by **empty stack** from a final state reachable from itself by popping  $Z_0$ . The final state is also reachable by popping  $S$ .

## Implementable?

Unfortunately, **no**. The described PDA **non-deterministically** selects handles and starts (one of potentially several) Reduce-transition-sequences or continues to Shift.



# Why non-determinism?

## Reduce-Reduce conflicts

These occur when the top of the stack corresponds to the **handle for two different rules**. The parser cannot deterministically choose one.

## Shift-Reduce conflicts

These occur when the top of the stack corresponds to some handle, but the parser **can continue to shift too**. It cannot deterministically choose whether to reduce now or do so later.

## A good question: if I shift, will I still be able to reduce?

In the arith.-expr. example, if we had shifted  $+$  before reducing  $\text{Id}$ , we would **never have gotten an accepting run**.

- The only handle that includes  $\text{Id}$  is  $\text{Exp} \rightarrow \text{Id}$ .

# Prefixes on the stack

## Proposition (Sentential-form prefixes on the stack)

Let  $(q, w, \gamma Z_0)$  be a configuration reached along an accepting run of the bottom-up parser with  $q$  not of the form  $(A, \dots)$ . Then:

$$S \Rightarrow^* \gamma^R \cdot w$$

*along a rightmost derivation.*

## Be careful!

The above **does not mean** that if we make sure the stack always has prefixes of valid sentential forms then we will get an accepting run!

- $\text{Id}+$  is a sentential-form prefix (see previous example)

# Viable prefixes on the stack

## Definition (Right-sentential form)

A sentential form that can be derived via a right-most derivation.

## Definition (Viable prefix)

A prefix  $\pi$  is **viable** if it is the prefix of a right-sentential form that can occur (in reverse) on the top of the stack along an accepting run of the bottom-up parser.

- There is an accepting run with a configuration  $(q, w, \pi^R Z_0)$  with  $q$  not of the form  $(A, \dots)$ .

# Recognizing viable prefixes

## Definition (The canonical finite-state machine)

The CFSM  $A_G$  is an automaton that recognizes as its language exactly the set of all viable prefixes of a given CFG  $G$ .

|     |     |               |       |
|-----|-----|---------------|-------|
| (1) | $S$ | $\rightarrow$ | $A\$$ |
| (2) | $A$ | $\rightarrow$ | $aCD$ |
| (3) |     | $\rightarrow$ | $ab$  |
| (4) | $C$ | $\rightarrow$ | $c$   |
| (5) | $D$ | $\rightarrow$ | $d$   |

## Exercises

- What is the language of the above grammar?
- Is the grammar LL(1)?
- What is its set of viable prefixes?

# Items and their closures

## Definition (Item)

An **item** of a grammar is a rule where we have inserted  $\bullet$  as a marker somewhere in the RHS, e.g.  $A \rightarrow \alpha_1 \bullet \alpha_2$ .

## Definition (Closure of an item)

The **closure of an item**  $A \rightarrow \alpha_1 \bullet B \alpha_2$  with  $B \in V$  is the set of all items  $B \rightarrow \bullet \beta$  where  $B \rightarrow \beta$  is a rule of the grammar.

- The closure of a set  $I$  is **the minimal closure-closed set that includes  $I$** .

# The CFSM (finally!)

## CFSM states

The states of our CFSM will be **sets of items**. The  $a$ -successor of a state  $I$ , for  $a \in T \cup V$ , is **the closure** of the set

$$\{A \rightarrow \alpha_1 a \bullet \alpha_2 \mid A \rightarrow \alpha_1 \bullet a \alpha_2 \in I\} .$$

## Definition (The CFSM)

The CFSM is the DFA  $(Q, V \cup T, \delta, q_0)$  where

- $Q$  is the set of all subsets of items,
- $q_0$  is the closure of  $\{S \rightarrow \bullet \alpha \mid S \rightarrow \alpha \text{ is a rule}\}$ ,
- $\delta(q, a)$  maps to the closure of the  $a$ -successor of  $q$ .
- (All states are accepting! Except for the empty set, i.e. a bad sink.)

# CFSM properties

## Theorem

*The CFSM recognizes the set of all viable prefixes.*

## Exercise

Construct the CFSM for the grammar from the previous exercises.

# LR(0) parsers

## What are they?

They are Left scanning, Right parsing parsers.

## Main intuition for LR(0)

We simulate the bottom-up PDA and, to resolve Shift-Reduce conflicts, we use the CFSM: From every configuration reached we run<sup>a</sup> the CFSM on the (reversed) contents of the stack.

- If the state reached contains an item  $S \rightarrow \alpha \bullet$ , then the parser should accept.
- If the state reached contains an item  $A \rightarrow \alpha \bullet$  then we have read a full handle and we Reduce  $A$ .
- Otherwise, we Shift.

---

<sup>a</sup>To be precise: this is done on the fly



# CFSM-based stack and action table

## Exercise

- Simulate the LR(0) parser for  $acd\$$  and the previous grammar.

## Parsing based on the CFSM

What is the LR(0) parser really doing?

- For every stack configuration, we get a unique state of the CFSM.
- The CFSM helps us determine whether to Reduce or Shift. (We can build a Reduce-Shift-Accept **action table** for the CFSM.)
- Conflicts are explicit in the CFSM state:
  - **Reduce-Reduce** if it contains items  $A \rightarrow \alpha\bullet, B \rightarrow \beta\bullet$ ;
  - **Shift-Reduce** if it contains items  $A \rightarrow \alpha\bullet, B \rightarrow \beta_1 \bullet \beta_2$ .
- So why not **push states of the CFSM** instead of symbols into the stack?

# The action table for LR(0) parsers

## The action table

Index grammar rules  $1 \leq j \leq n$  and states from the CFSM  $0 \leq i \leq k$ . The table  $T$  maps each  $i$  to a set of actions:

- $T(i)$  contains (a Reduce)  $j$  if state  $i$  has the item  $A \rightarrow \alpha \bullet$  with  $A \rightarrow \alpha$  the  $j$ -th rule.
- $T(i)$  contains Shift if state  $i$  has an item  $A \rightarrow \alpha_1 \bullet \alpha_2$ .
- $T(i)$  contains Accept if state  $i$  has an item  $S \rightarrow \alpha \bullet$ .
- $T(\emptyset)$  contains an Error action only.

## Deterministic

Note that every cell of the action table has at least one action because all states from the CFSM contain some item (except  $\emptyset$ ).

- The parser is deterministic if every cell has exactly one entry.
- If the latter holds, the grammar is LR(0).

# Action table with a stack of states

## Algorithm

1. Push the initial state 0 of the CFSM into the stack  $\mathcal{S}$ .
2. As long as we can't accept or get an error:
  - 2.1 If  $j \in T(\text{top}(\mathcal{S}))$ , where  $A \rightarrow \alpha$  is the  $j$ -th rule, then
    - 2.1.1 pop  $|\alpha|$  times from  $\mathcal{S}$ , where  $A \rightarrow \alpha$  is the  $j$ -th rule,
    - 2.1.2 and put  $A$  in variable  $c$ .
  - 2.2 Otherwise, put the next symbol into variable  $c$ .
  - 2.3 Let  $q'$  be the next state  $\delta(\text{top}(\mathcal{S}), c)$  of the CFSM after reading  $c$ .
  - 2.4 Push  $q'$  into  $\mathcal{S}$ .